

General Developer Journey			Points		4		3		2		1		0		Score	Min Score	Max Score	Comment	Found Issues	
Phase	Action	Description	Great Experience	Good Experience	Neutral Experience	Bad Experience	Terrible Experience													
Phase 1: Environment Setup & Sanity Check																				
Development Environment Setup	Install TT-NN and run basic examples		One-click setup with containerized environments, automatic dependency resolution, and integrated development tools	Comprehensive setup guide with automated scripts, clear dependency management, and troubleshooting sections.	Standard setup documentation with most dependencies covered, but some manual configuration still required for optimal performance.	Basic setup instructions exist but are outdated or incomplete, leading to version conflicts and missing dependencies.	No setup documentation. Developers must figure out dependencies, versions, and configuration from scattered sources.								0		4			
	Verify Installation	Run basic operations tutorial	Script runs first try	Works after minor fixes	Several attempts needed	Multiple errors encountered	Cannot get basic script working								0		4			
Phase 2: Implementation																				
Tensor Layout and Memory Management	Understanding TT-NN's tensor layouts		Intuitive interface that automatically suggests optimal layouts based on operation types, with comprehensive performance benchmarks.	Clear documentation with performance guidelines and multiple examples showing layout choices for different operation types.	Documentation covers the basics with some examples, but you still need significant experimentation to optimize for their specific use case.	Basic documentation exists but lacks practical examples. The options are listed but it is not clear how they affect performance and use cases.	No clear documentation on when to use ROW_MAJOR vs TILE. Forced to reverse-engineer from source code or trial-and-error, leading to longer than expected experimentation on based on error messages.								0		4			
	Managing tensor shapes and padding		Automatic reshaping and padding with performance warnings when suboptimal choices are made.	Clear documentation with best padding choices explained and helpful error messages that suggest corrections.	Basic padding requirements are documented, but optimal strategies for performance are unclear.	Some documentation exists but is scattered. Error messages are unclear, such as, "Invalid tensor shape", but without guidance on how to fix.	Padding requirements are undocumented, leading to runtime crashes with no clear error messages. Excessive time is wasted figuring out alignment requirements.								0		4			
	Memory allocation		Automatic memory management.	There is clear documentation on data allocation and movement, with tradeoffs explained in sufficient detail.	Basic data movement is documented, and there are enough examples to get by, but choices are not explained.	Basic memory management must be managed manually with minimal guidance or feedback.	No visibility into memory usage, frequent out-of-memory crashes with no guidance on optimization strategies.								0		4			
	Framework Translation Challenges	Converting from PyTorch/TensorFlow patterns		Migration is straightforward and easy using similar or identical flows	Comprehensive migration guides for common cases.	Decent operation mapping with some pattern examples, but gaps exist for advanced use cases.	Basic operation mapping exists but complex patterns (like custom layers) have no guidance for translation.	No migration guides or equivalent operation mappings. Conversion from scratch with completely different paradigms.								0		4		
		Handling unsupported operations		Automatic decomposition of unsupported operations with performance warnings and optimization suggestions.	Comprehensive operation support with detailed decomposition guides and performance implications.	Clear support matrix with basic decomposition examples for common unsupported operations.	Supported operations are listed but decomposition strategies for unsupported ones are undocumented.	No clear list of supported vs unsupported operations. Discover limitations at runtime with no workaround guidance.								0		4		

Model Conversion Workflow	PyTorch to TT-NN conversion sequence	Conversion for most models is straightforward with intelligent handling of edge cases and clear reporting of any manual steps needed.	Well-documented conversion pipeline with for common architectures and clear guidance for edge cases.	Standard workflow exists with examples, but complex models require significant manual intervention.	Basic conversion steps documented but many edge cases and are not, leading to frequent failures.	No established workflow. Explore entire conversion pipeline from scratch, often discovering incompatibilities late in the process.	0	4
	Weight conversion and initialization	Automatic weight conversion with integrity checking and performance optimization during conversion.	Comprehensive weight conversion within operations and support for all major formats.	Conversion relatively straightforward with most common formats, but some manual work required for complex cases.	Poorly documented and not clear how to handle all PyTorch weight formats.	Developers must manually handle format conversions with no validation of correctness.	0	4
	Input preprocessing pipelines	Automatic input preprocessing with optimal performance.	Comprehensive preprocessing with easy integration patterns and performance optimization.	Standard preprocessing operations supported with some integration examples.	Basic preprocessing operations available but integration with existing pipelines is unclear.	No guidance on how to handle preprocessing. All preprocessing must be done from scratch in TT-NN format.	0	4
API Learning Curve	Understanding TT-NN programming approach	Interactive tutorials and clear conceptual bridges that make the transition intuitive for developers from other frameworks.	Comprehensive guide comparing approaches with clear migration patterns and best practices.	Decent explanation of functional approach with some comparative examples to other frameworks.	Basic functional concepts explained but practical application to deep learning workflows is unclear.	No conceptual documentation. PyTorch developers are completely lost trying to understand the functional paradigm without guidance.	0	4
	Learning function signatures and parameters	TT-NN API matches PyTorch with near-identical usage patterns.	Comprehensive API documentation with detailed examples and parameter impact explanations.	Standard API documentation with basic examples, but complex parameter interactions are unclear.	Basic API reference exists but lacks examples and parameter descriptions are minimal.	No API documentation. It is necessary to read source code to understand function signatures and parameter meanings.	0	4
	Error handling and edge cases	Intelligent error handling with automatic recovery suggestions and proactive edge case detection.	Comprehensive error handling guide with clear recovery patterns and prevention strategies.	Standard error handling patterns with some examples, but edge case handling is inconsistent.	Basic error types documented but recovery strategies and prevention methods are unclear.	No error handling guidance. Runtime failures provide no context about what went wrong or how to fix it.	0	4
Integration and Deployment	Device initialization and context management	Automatic device management with intelligent resource allocation and cleanup.	Straightforward device management with clear and comprehensive documentation.	Standard initialization patterns documented with basic context management, but advanced scenarios lack guidance.	Basic device initialization examples exist but documentation is lacking and more complex scenarios are not explained.	No guidance on device management.	0	4

Summary	Overall impact assessment	Prototyping and deployment is quick and easy to achieve. Default implementation produces good performance.	Solutions can be implemented efficiently with clear guidance, spending most time on actual model development rather than framework learning.	Complete the project with reasonable effort, but require significant learning time and some experimentation.	Eventually succeed but only after extensive trial-and-error, taking significantly longer than expected with high frustration levels.	Task is abandoned after significant frustration, finally being unable to get basic functionality working.	0	4
TT-NN Visualizer								
Phase	Action	Description	Great Experience	Good Experience	Neutral Experience	Bad Experience	Terrible Experience	
Phase 1: Environment Setup & Sanity Check								
Time Taken: How long did this phase take? Installation Issues: Did pip install work smoothly? Any driver/dependency errors? Sanity Check: Did ttnn_basic_operations.py run successfully first try? Clarity Score (1-5): How clear were initial setup instructions?	Install tt-metal with profiler and ttnn	Clone tt-metal repository, build tt-metal with profiler enabled, build ttnn from source to access built tt-metal	Instructions work smoothly	Minor dependency issues resolved	Some trial and error needed	Installation fails multiple times	Complete installation failure	0 4
	Verify Installation	Run basic operations tutorial	Script runs first try	Works after minor fixes	Several attempts needed	Multiple errors encountered	Cannot get basic script working	0 4
Phase 2: TTNN-Visualizer installation and verification								
Time Taken: How long did this phase take? Installation Issues: Did pip install work smoothly? Any driver/dependency errors? Sanity Check: Did ttnn-visualizer run successfully first try (opened website)? Clarity Score (1-5): How clear were initial setup instructions?	Install ttnn-visualizer	pip install ttnn-visualizer	Instructions work smoothly	Minor dependency issues resolved	Some trial and error needed	Installation fails multiple times	Complete installation failure	0 4
	Run ttnn-visualizer	ttnn-visualizer	Script runs first try - homepage shows up	Works after minor fixes	Several attempts needed	Multiple errors encountered	Cannot get basic script working	0 4
Phase 3: Profiling and gathering memory report data								
Profiling Success: Did profiling complete successfully without crashing or hanging? Clarity of Profiling Instructions (1-5): How clear and understandable were the steps to collect profiling and memory data? Sanity Check: Did the profiling output get generated and look reasonable on the first attempt? Data Quality: Was the output data detailed and accurate enough to be useful? Visualization Integration: Were the profiling results easy to visualize or import into tools like TTNN Visualizer? Error Feedback: Were error messages clear and actionable if profiling or reporting failed?	Set configuration file	Create a setup file and set the global variable to point to it	Config file loads correctly and profiler can start	Minor fixes needed to recognize config	Needs debugging to activate correctly	Profiler fails to recognize config	Config setup consistently fails	0 4
	Gather data	Collect memory data by running the target model with profiling configuration	Memory data is generated as expected	Minor issues when generating data	Data generated after several runs	Multiple profiling errors	Data not generated at all	0 4
	Collect created report	Collect created data from machine	File collected without issue	Minor directory confusion resolved	Took multiple attempts to locate	File not found at first	Could not locate report file	0 4
	View memory report	Upload collected data to the visualizer under the Memory reports section.	Upload successful, data displays correctly	Several tries needed	Needs re-export to work	Upload fails initially	Upload not possible	0 4
Phase 4: Profiling and gathering performance report data								
Profiling Success: Did profiling complete successfully without crashing or hanging? Clarity of Profiling Instructions (1-5): How clear and understandable were the steps to collect profiling and performance data? Sanity Check: Did the profiling output get generated and look reasonable on the first attempt? Report Usefulness: Was the report detailed and useful for understanding runtime performance (e.g., op latency, throughput)? Visualization Integration: Were the profiling results easy to visualize or import into tools like TTNN Visualizer? Error Feedback: Were error messages clear and actionable if profiling or reporting failed?	Gather performance data	Collect performance data by running model with Tracy profiler enabled	Tracy data collected as expected	Tracy data collected after config hweaks	Needed reruns to collect data	Multiple errors in profiler	No data collected	0 4
	Collect created report	Collect created data from machine	File collected without issue	Minor directory confusion resolved	Took multiple attempts to locate	File not found at first	Could not locate report file	0 4
	View performance memory report	Upload collected data to the visualizer under the Performance reports section.	Upload successful and data visualized	Minor fix to display data	Needs re-export to work	Viewer crashes or fails	Upload not possible	0 4
Phase 5: Analyze results								
	Review Memory & Performance Reports	Explore visualizations to identify performance bottlenecks and data movement issues	Clear bottlenecks identified, leads to optimization ideas	Some areas of concern identified	Needed deeper manual analysis	Difficult to interpret data	Visualization unclear or broken	0 4

